

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

5 SLOT-D

Course Code:	CSA1407	Course Title:	COMPILER DESIGN
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	40% of CIA		
CO1 Statement:	Apply lexical analysis for token specification and recognition using LEX tool. (BL3)		
Student Name:	BALAJI.K	Reg. No.:	192324255
Section / Batch:		Date:	16/07/2026

Code of Conduct

I BALAJI.K/192324255 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: [Signature]

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	4 / 4	4 / 4	3 / 4	4 / 4	3 / 4	90
2	4 / 4	3 / 4	3 / 4	4 / 4	4 / 4	88.75
3	3 / 4	4 / 4	3 / 4	4 / 4	4 / 4	88.75
4	4 / 4	4 / 4	3 / 4	3 / 4	3 / 4	85
5	3 / 4	3 / 4	4 / 4	4 / 4	4 / 4	90
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	88.5
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

N. D. J. P.

Annexure A

ANALYTICAL PROBLEM SOLVING

1	Trace the processing of the input expression $P = (a * b) + (b * c) * 2$ through all phases of a compiler. - Identify and list all tokens generated during lexical analysis. - Construct the syntax tree during the parsing phase. - Generate the intermediate code for the given expression. - Apply suitable optimizations to the intermediate code. - Explain the significance of the final target code prod
2	For each of the following Regular Expressions, construct the language. Write any FIVE valid strings for each Regular Expression. $(\epsilon + aa^*)^*$ $(a^* + b^*)^* abb$ $(a^*b^*)^* aba (a^* + b^*)^*$ $(a + ab)^*$ $(aba)^* + (a^*b^*)^*$
3	i) Find the number of tokens in the following C statement is <pre>main() { int a=10; char b ="abc"; in t c = 30; ch ar d = "xyz"; in /* comment */ t m = 40.5; }</pre> ii) Identify the lexical errors in the following C statement is <pre>void main() { int x=10, y=20; char * a; a= &x; x= 1xab; }</pre>
4	Find a regular expression corresponding to each of the following subsets of $\{0, 1\}^*$: - The language of all strings that have an even number of 0's. - The language of all strings that contain at least one 1. - The language of all strings in which both the number of 0's and the number of 1's are odd. - The language of all strings that start with 1 and end with 0. - The language of all strings that do not contain consecutive 1's.

5 Consider a lexical analyzer for a simplified programming language. The language consists of the following tokens:

1. Keywords: if, else, while, return, int, float
2. Operators: +, -, *, /, =, ==, <, >
3. Separators: (,), {, }, :, ,
4. Identifiers: A sequence of letters followed by a sequence of digits, e.g., var123
5. Integer Literals: A sequence of digits, e.g., 12345
6. Floating-point Literals: A sequence of digits followed by a dot and another sequence of digits, e.g., 123.45

The lexical analyzer needs to tokenize the following input string:
int var123 = 12345; float var456 = 123.45; if (var123 == var456) { return; }

Questions:

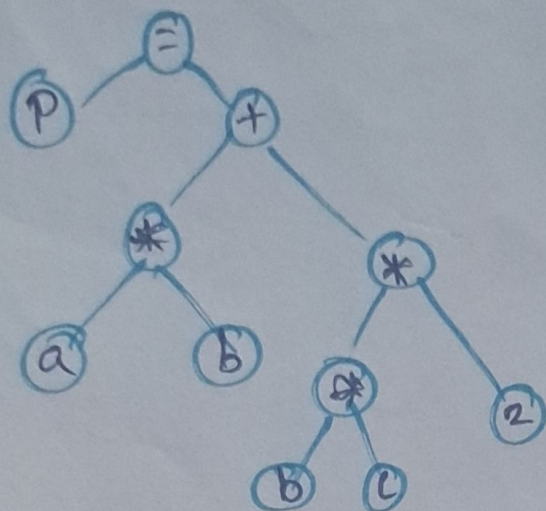
- a) How many tokens are there in the given input string?
- b) Provide a list of tokens categorized by their types (keywords, operators, separators, identifiers, integer literals, floating-point literals).
- c) Assuming a finite state machine (FSM) is used for lexical analysis, estimate the number of state transitions required to tokenize the given input string. Provide your reasoning based on the transitions needed for each token type.

$$p = (a * b) + (b * c) * 2$$

i) tokens

$\langle id \rangle \langle = \rangle \langle (\rangle \langle a \rangle \langle * \rangle \langle b \rangle \langle + \rangle \langle (\rangle \langle b \rangle \langle * \rangle \langle (\rangle \langle \rangle \rangle \langle + \rangle \langle 2 \rangle$

ii) Syntax tree:



iii) Intermediate code

$$t_1 = b * c$$

$$t_2 = \text{int to real } 2.0$$

$$t_3 = t_1 + t_2$$

$$t_4 = a * b$$

$$t_5 = t_4 + t_3$$

$$p = t_5$$

iv) Optimization of Intermediate code.

$$t_1 = b * c$$

$$t_2 = t_1 * 2$$

$$t_3 = a * b$$

$$p = t_2 + t_3$$

v) Code generator.

LOAD R0, b

MUL R0, c

MUL R0, #2

LOAD R1, a

MUL R1, b

ADD R1, R0

STORE p, R1

2) For each of the following construct the language write any 5 valid strings.

a) $(\epsilon + a^*a^*)^*$

ϵ
a a
a a a
a a a a
a a a a a

b) $(a^* + b^*)^* abb$

abb
aabb
babb
aaabb
bbabb

c) $(a^*b^*)^*aba(a^* + b^*)$

aba
aaba
ababa
aaba
aaba

d) $(atab)^*$

ϵ
a
ab
aa
a ab

e) $(ab^*a)^* + (a^*b^*)^*$

ϵ
a ba
a b
a b a a a
a a a b b b

Find The number of tokens in the following statement

```
int main()
```

```
{ int a=10;
```

```
char b="abc";
```

```
int c=30
```

```
char d="xyz";
```

```
int /* comment */ m=40*5; }
```

S.NO	token	type
1.	main	keyword
2.	{	operator
3.	}	operator
4.	int	keyword
5.	a	Identifier
6.	=	operator
7.	10	integer
8.	,	operator
9.	char	keyword
10.	b	Identifier
11.	=	operator
12.	"abc"	string
13.	;	operator
14.	int	keyword
15.	c	Identifier
16.	=	operator
17.	30	integer
18.	,	operator
19.	char	keyword
20.	d	Identifier
21.	=	operator
22.	"xyz"	string
23.	;	operator
24.	int	keyword
25.	m	Identifier
26.	=	operator
27.	40*5	integer
28.	;	operator
29.	}	operator
30.		

2. Q1) Find lexical error in the program

```
void main()
```

```
{  
    int x=10, y=20;  
    char *a;  
    a = &x;  
    x = 1 xab; }  
}
```

Error Analysis

lexical error details:

Q1) Error Line: `x = 1 xab;`
Invalid token: `1 xab`

Attributes:

→ token: `1 xab`

→ type: Invalid Identifier

In C, identifiers must begin with letter or an underscore, not a number.

Q2) `a = &x;`

→ will trigger a semantic type mismatch error during compilation because you are assigning an `int*` to `char*` → This lex successfully generate valid tokens.

AJ

i) The language of all strings that have an even number of 0's

$$(1^* 01^* 01^*)^* 1^*$$

ii) The language of all strings that contain at least one 1.

$$(011)^* 1 (011)^*$$

iii) The language of all strings in which both the number of 0's and number of 1's are odd

$$(01110)(00111)1(01110)(01110)^* (01110)$$

iv) The language of all strings that start with 1 and end with 0.

$$1(011)^* 0$$

v) the language of all strings that do not contain consecutive 1's

$$(0110)^* (1/\epsilon)$$

5) Dnt var123=12345;

float var456=123.45;

Pf(var123==var456)

{
return;}
}

a) how many tokens are there in the given input string.

1. Dnt

2. var123

3. =

4. 12345

5. float

6. var456

7. =

8. 123.45

9. ;

10. Pf

11. {

12. }

13. var123

14. =

15. var456

16. >

17. {

18. Return

19. ;

20. }

} total
tokens = 20

3. Categorize The tokens.

Token:

int

var123

=

12345

;

float

var456

.

123.45

;

if

{

var123

= 2

var456

}

{

return

;

}

type:

keyword

identifier.

operator

Integer

separator

keyword

Identified.

operator

floating point literal

separator

keyword

separator

Identifier

operator

Identifier

separator

separator

keyword

separator

Operator.